

DOCKER

DOCKER

REFERENCES -

<https://projects.100xdevs.com/tracks/docker-2/docker-2-1/>

<https://petal-estimate-4e9.notion.site/Deploying-a-monorepo-to-VMs-19c7dfd10735808d9c2ae833fd2f2546>

VIRTUALIZATION

Imagine you have **one powerful computer**, but you want to **split it into many smaller computers** (virtual machines) that can **run different apps or operating systems (Windows, Linux, etc.)** at the same time.

➡ **Virtualization** allows you to create **multiple "virtual computers"** on a **single physical machine**.

Each virtual machine (VM) acts like a **real computer**, but it's just **software** running inside your actual computer.

A **Hypervisor** is like a **manager or referee** that creates and runs these virtual machines.

➡ The Hypervisor **shares** the physical computer's **CPU, memory, storage** among the virtual machines and makes sure they **don't disturb each other**.

Imagine you want to **run many apps** on a computer, but **don't want the overhead of creating whole virtual machines** (with full operating systems).

➡ **Containerization** puts each app in a **"container"** with just the necessary files and settings, so it's **lightweight** and **fast** to start.

Containers **share** the same operating system (unlike VMs), but they **run independently** like small boxes.

When we run **multiple apps** (or environments) on **one machine**, we need to make sure that:

- Everyone (app, VM, or container) **gets enough memory, CPU, disk, etc.** to work properly.
- They **don't interfere** with each other.
- The system doesn't **run out of memory** or crash
- **Virtualization** creates **full virtual machines (VMs)** — like separate computers inside your real computer.
- Each VM has its **own dedicated chunk of memory, CPU, and disk** space.
- **Hypervisor** (the software that runs VMs, e.g., VMware, VirtualBox) manages these resources.

How does Hypervisor manage resources?

- **Pre-allocates RAM:** When you start a VM, the hypervisor gives it a fixed amount of memory (e.g., 4 GB out of your 16 GB RAM).
- **Shares CPU:** Hypervisor divides CPU time between VMs.
- **Virtual Disk Space:** Each VM gets a virtual hard drive (a big file on your real hard drive).
- Hypervisor prevents VMs from **using too much RAM or CPU** so others don't get starved.

By default, containers share available memory and CPU with others. Containers do not share memory directly, each container runs its own application, and by default they can't see or access each other's RAM.

Virtual Machines (VMs)

Virtual machines create complete, isolated computer systems on top of a host operating system. Each VM includes its own full operating system, kernel, drivers, and allocated hardware resources like CPU, memory, and storage. A hypervisor manages these VMs and provides the abstraction layer between the physical hardware and virtual systems.

Containers

Containers share the host operating system's kernel but isolate applications at the process level. They package an application with its dependencies, libraries, and

configuration files into a lightweight, portable unit. Container engines like Docker manage these isolated processes.

To install docker go to → <https://docs.docker.com/desktop/setup/install/mac-install/>

and install docker for mac with apple silicon, and check it in the terminal,

`docker` if the command works then the docker is installed in the system!

CONTAINERIZATION

Imagine u built a cool game on your computer and you want to share it with your friends so they can play or use it on their computers.

But here is the problem,

- Your game needs special software to run
- it only works on certain computers

Docker is like a magical box(container) maker. It packs ur app/game with everything it needs like code,software and puts them in a container.

A container is like a special package that runs the same way on any computer - windows,mac or linux or even server.

Key concepts related to docker

WRITING A DOCKER FILE

LAYERS

VOLUMES AND NETWORKS

DOCKER COMPOSE

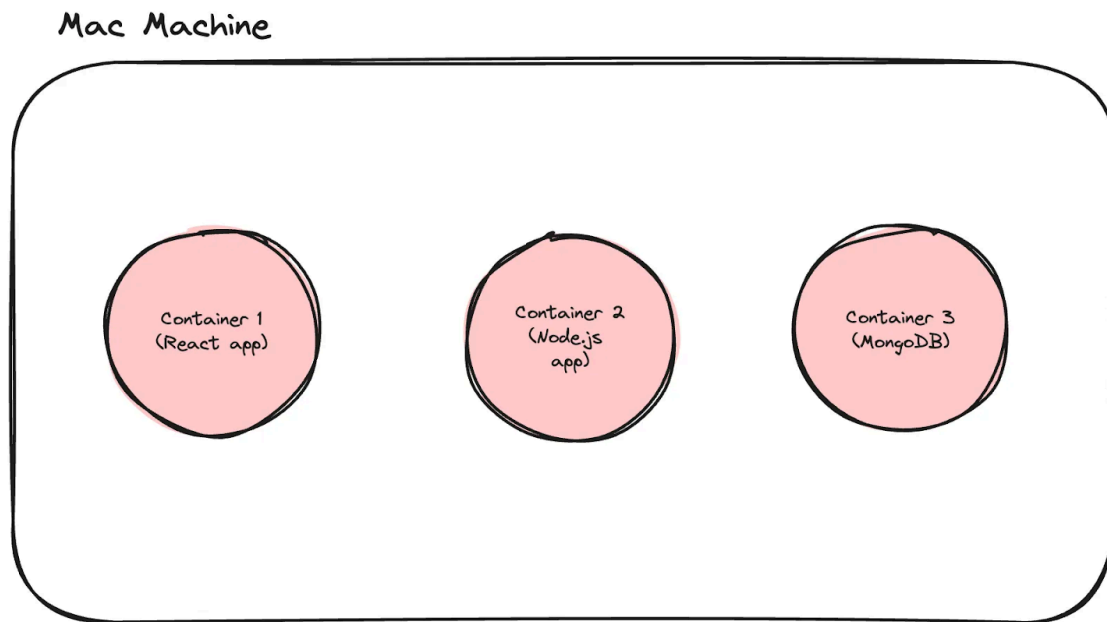
Optimizing Docker images

Docker in Development

Multi-stage builds

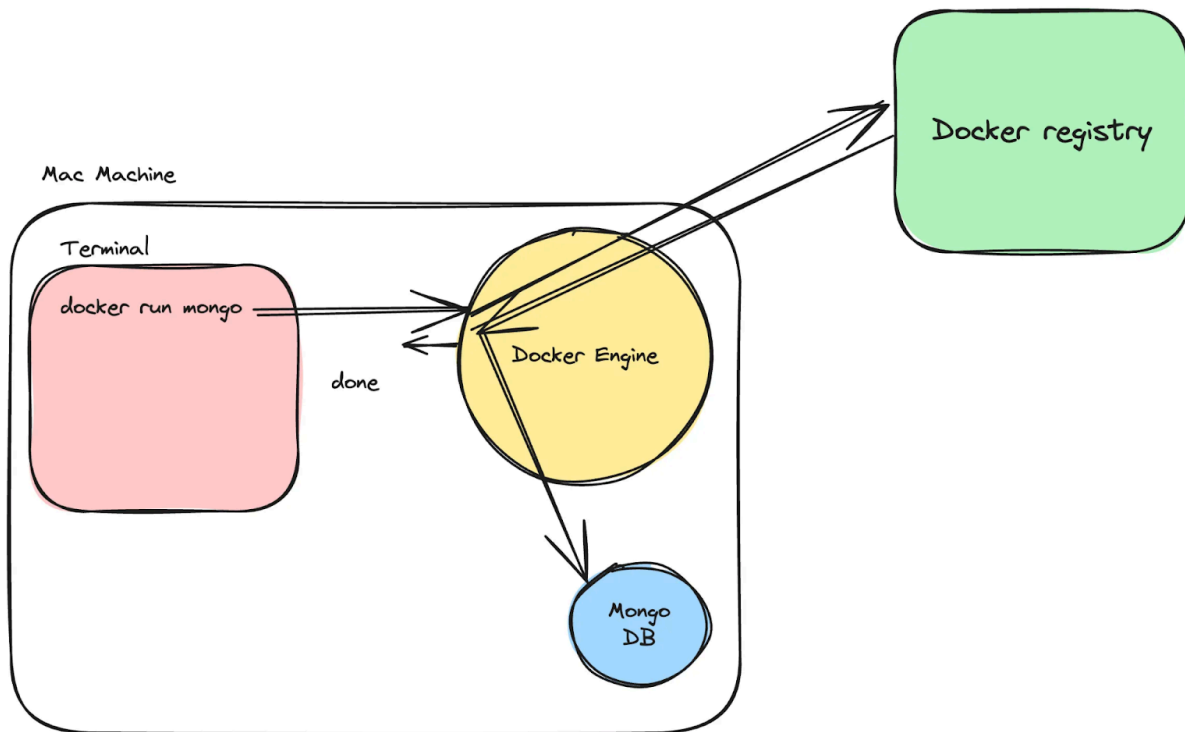
.Containers

Docker container has its own file system as well.



If u have a container, where ur codebase, then your friend can just pull that container, and execute the codebase, without necessarily configuring the files and installing dependencies.

Install the docker,



2.images -

A **Docker image** is like a **recipe** for creating containers.

It tells Docker **how to build the container** and **what to put inside**.

You can **reuse** and **share** images!

Image has its own OS,FILES, source code,

An image in execution is a container.

`docker ps` // it will show u all the containers

3.Dockerfile

A **Dockerfile** is a file that has **instructions** for building a Docker image.

It tells Docker:

- What base to start from (like Linux).
- What software to install.
- What command to run when starting the app.

docker build // command to create own image,

How do u containerize a nodejs application?

Start with a predefined image,

FROM ubuntu

// the first line of the docfile, will start from a basic image/existing image and via this we create a fresh ubuntu, install nodejs

OR

FROM node:22

// u can also start from base image of node, rather than starting from ubuntu,

WORKDIR /app

// working directory, where do u want to store all the codes

COPY ./index.js ./index.js

//copy index.js file to index.js file in the nodejs container root dir app folder

COPY . .

// copy all the modules to the container

.dockerignore - same as .gitignore, it will ignore node_module folder from copying the folder in the nodejs container,

Let's say in .dockerignore file there is,

node_modules

RUN npm install

RUN npm run build

//run the above commands to build the code

EXPOSE 3000

//expose a port 3000

CMD ["node" , "index.js "]

// can only have one cmd file, when a container runs, what should be run is whats written on CMD

The above CMD will run when you write

```
docker run -p 3000:3000 harkiratproject
```

Now, how to build a newly created image, run the below command,

```
docker build -t hello-world .
```

What is the dot at the last?

If u want to pass environment variables, then u can do

```
docker run -p 3000:3000 -e DATABASE_URL=zia@123 hello-world
```

USES IN FULL STACK